

Módulo 4

Estruturas de Dados Estáticas

Strings

Strings são usadas para armazenar texto/caracteres.

Por exemplo, "Hello World" é uma sequência de caracteres.

Ao contrário de muitas outras linguagens de programação, C não possui um tipo String para criar facilmente variáveis de string.

```
int main() {  
    char greetings [ ] = "Hello World!";  
    return 0;  
}
```

Strings

Observe que devemos usar aspas duplas ("").

Para gerar a string, você pode usar a função **printf()** junto com o especificador de formato **%s** para informar ao C que agora estamos trabalhando com strings.

```
#include <stdio.h>

int main() {
    char greetings [ ] = "Hello World!";
    printf("%s", greetings);
    return 0;
}
```

Posição de acesso

Como strings são na verdade arrays em C, podemos acessar uma string referindo-se ao seu número de índice entre colchetes [].

Este exemplo imprime o primeiro caractere [0] em saudações.

```
#include <stdio.h>

int main() {
    char greetings [ ] = "Hello World!";
    printf("%c", greetings[0]);
    return 0;
}
```

Modificar strings

Para alterar o valor de um caractere específico em uma string, consulte o número do índice e use aspas simples (' ').

```
#include <stdio.h>

int main() {
    char greetings [ ] = "Hello World!";
    greetings [0] = 'J';
    printf("%c", greetings[0]);
    return 0;
}
```

Ciclo através de uma string

Podemos percorrer os caracteres de uma string, usando um ciclo **for**.

```
#include <stdio.h>

int main() {
    char carName [ ] = "Volvo";

    for (int i = 0; i < 5; i++ ) {
        printf( "%c\n" , carName [i]);
    }

    return 0;
}
```

Outra maneira de criar strings

Devemos observar que podemos criar uma string com um conjunto de caracteres.

```
#include <stdio.h>

int main() {
    char greetings [ ] ={'H', 'e', 'l', 'l', 'o', '!', '\0'};
    printf("%s" , greetings);

    return 0;
}
```

Outra maneira de criar strings

Por que incluímos o **'\0'** personagem no final?

```
int main() {  
    char greetings [ ] = { 'H', 'e', 'l', 'l', 'o', '!', '\0' };  
    printf("%s" , greetings);  
    return 0;  
}
```

Isso é conhecido como **caractere de terminação nulo** e deve ser incluído ao criar strings usando este método. Este é o fim da string.

Diferenças

A diferença entre as duas formas de criar strings, é que o primeiro método é mais fácil de escrever, e não precisamos incluir o caractere, **'\0'** pois C fará isso por nós.

```
int main() {  
    char greetings [ ] = "Hello!";  
    char greetings2 [ ] = { 'H', 'e', 'l', 'l', 'o', '!', '\0' };  
  
    printf("%d\n", sizeof(greetings));  
    printf("%d\n", sizeof(greetings2));  
    return 0;  
}
```

Exemplo prático

```
#include <stdio.h>
#include <string.h>

int main() {

    char message [ ] = "Good to see you, ";
    char fname [ ] = "John";

    printf("%s %s!", message, fname);

    return 0;
}
```

Desafio

1. Peça ao utilizador que introduza um nome completo.
2. Utilize um ciclo para percorrer cada carácter do nome e imprima-os individualmente, um por linha.
3. Modifique a string, alterando todas as vogais para maiúsculas. Por exemplo, se o utilizador inserir "Olá", a saída deve ser "OlÁ".
4. Inverta a string, de modo que a última letra se torne a primeira, a penúltima se torne a segunda e assim por diante. Por exemplo, se o utilizador inserir "mundo", a saída deve ser "odnum".
5. Imprima a string resultante após as modificações.

Desafio

Insira um nome: Oleh

Caracteres individuais:

O

l

e

h

String com vogais em maiúsculas: OLEh

String invertida: Helo